Fallstudien II / Case Studies II

Accuracy of Surrogate Modeling of a Resource-Heavy Physics Simulation

with the case of using Single-Layer Neural Networks

Supervisors:

Prof. Dr. Katja Ickstadt

Dr. Henrike Weinert

Yassine Talleb

Carmen van Meegen

Authors:

Huseyin Mert Ezer (270981)

Sila Aydin (Group Member)

Elif Ilgin Yildiz (Group Member)

Date:

January 12, 2026

Contents

1 Introduction **4**

1.1 Motivation & Project Goal 4

1.2 Approach 4

1.3 Overview & Key Results 4

2 Description of the Problem **5**

2.1 Objectives 5

2.2 Data Material 5

2.2.1 Experimental Design and Sample Size 5

2.2.2 Input Variables 6

2.2.3 Response Variable 6

2.2.4 Data Quality and Epistemological Context 6

3 Statistical Methods **6**

3.1 Data Preprocessing 7

3.2 Surrogate Model Architecture 7

3.3 Parameter Estimation and Regularization 7

3.4 Model Selection and Baseline Comparison 8

3.5 Uncertainty Quantification (Deep Ensembles) 8

3.6 Sensitivity Analysis 9

3.7 Data Efficiency Analysis (Learning Curve) 9

4 Statistical Analysis **9**

4.1 Model Selection and Performance 9

4.2 Verification of Assumptions 10

4.3 Sensitivity Analysis and Physical Interpretation 10

4.4 Data Efficiency and Convergence 11

5 Summary **11**

5.1 Project Review 11

5.2 Summary of Results 12

5.3 Discussion 12

5.4 Outlook 12

A R Source Code **14**

B Additional Plots **43**

C Tables **46**

1 Introduction

1.1 Motivation & Project Goal

Finite Element Methods (FEM) provide high-fidelity simulations for structural analysis but are computationally expensive. In the context of the sun sail structure analyzed in this project, running extensive simulations using the Kratos Multi-Physics software is resource-intensive, making it unsustainable for tasks requiring frequent evaluations. This report addresses the need for a computationally efficient surrogate model that can mimic the output of the original deterministic simulation while minimizing computational costs.

The primary objective is to approximate the maximum membrane stress ($\sigma_{mem,max}$) of the sun sail structure, which serves as the critical failure criterion. By constructing a surrogate model based on a limited set of $N = 144$ observations derived from a stratified design, we aim to quantitatively and qualitatively analyze the accuracy and suitability of neural networks for this regression task.

1.2 Approach

To solve this approximation problem, we employ a Single-Layer Neural Network (SLNN) as the candidate surrogate model. The choice of an SLNN is theoretically justified by the Universal Approximation Theorem, which states that standard feedforward networks with as few as one hidden layer are capable of approximating any continuous function on a compact set of inputs (Hornik et al., 1989).

Given the data scarcity inherent in the project budget ($n_{max} \leq 200$) and the non-convex nature of the neural network objective function, we implement a rigorous statistical framework. We utilize Leave-One-Out Cross-Validation (LOO-CV) to maximize the training data utility and employ a Deep Ensemble strategy (Lakshminarayanan et al., 2017) to stabilize predictions and quantify epistemic uncertainty. Furthermore, we verify the physical validity of the model through sensitivity analysis using Permutation Feature Importance (PFI).

1.3 Overview & Key Results

The analysis reveals that while the Single-Layer Neural Network achieves exceptional predictive accuracy ($Q^2 \approx 0.999$) on the provided dataset, it operates in a "Data Hungry" regime where the current sample size ($N = 144$) is insufficient for full statistical convergence. The model demonstrates strong physical validity, correctly identifying membrane pre-stress and surface load as dominant factors. However, reliability analysis via Deep Ensembles indicates a tendency toward overconfidence, with coverage probabilities (76.4%)

falling below the theoretical 95% target due to the minimal regularization employed.

The remainder of this report is structured as follows: **Section 2** describes the Full Factorial Design and the variables involved. **Section 3** details the statistical methodology, including the SLNN architecture and the Deep Ensemble framework. **Section 4** presents the quantitative results, including hyperparameter selection, sensitivity metrics, and learning curves. Finally, **Section 5** summarizes the study's conclusions and offers an outlook for future improvements.

2 Description of the Problem

2.1 Objectives

The primary objective of this project is to evaluate the accuracy and suitability of a statistical surrogate model intended to replicate the resource-intensive Finite Element Method (FEM) simulations. From a content perspective, the goal is to predict the structural stress behavior of a sun sail under varying conditions. From a statistical perspective, the objective is to perform a rigorous quantitative and qualitative analysis of the surrogate's performance, specifically characterizing its predictive capability and robustness in the absence of stochastic noise.

2.2 Data Material

2.2.1 Experimental Design and Sample Size

The data collection strategy employs a planned computer experiment rather than an observational study. To ensure comprehensive coverage of the input space within the project's computational budget ($n_{max} \leq 200$), a **Full Factorial Design** was employed.

Unlike a simple random sample, this design systematically tests all possible combinations of the factor levels to ensure orthogonality, making the effects of individual parameters easier to recognize. The number of levels for each parameter was determined by its Coefficient of Variation (CV). The two parameters with the highest variability (f_{mem} and σ_{mem}) were sampled at three levels (10th, 50th, and 90th percentiles), while the remaining four parameters were sampled at two levels. This configuration results in a total sample size of:

$$N = 3^2 \times 2^4 = 144 \text{ simulations} \tag{1}$$

which fits well within the specified budget.

2.2.2 Input Variables

The simulation depends on a set of $d = 6$ uncertain input parameters representing the material properties and loading conditions. As specific empirical data for these materials was not available, we utilize **assumed theoretical distributions** derived from the initial FEM study (Holcher, 2020). These inputs are summarized in Table 1 (see Appendix).

2.2.3 Response Variable

The scalar output of interest (response variable y) is the **Maximum Cauchy stress** in the membrane ($\sigma_{mem,max}$). This metric is critical for reliability analysis. The structural integrity is assessed by comparing this output against the material's Rupture Stress ($\sigma_{mem,y}$), though the rupture stress itself acts as a threshold parameter rather than an input to the regression model.

2.2.4 Data Quality and Epistemological Context

The dataset possesses distinct qualities characteristic of computer experiments that differentiate it from physical surveys:

1. **Completeness:** Since the experimental design is manually constructed via the Full Factorial method, there are no missing values in the dataset.

2. **Deterministic Output:** The data generation process is a deterministic computer simulation. Unlike physical measurements, the output is free from aleatory uncertainty (random measurement noise); the same input set $\mathbf{x}$ will always yield the exact same response y.

3. **Recursive Simulation:** There is an epistemological nuance to this experiment. The FEM simulation is itself a mathematical approximation of a physical object. Consequently, the surrogate model represents a second layer of abstraction—a simulation of a simulation. The "accuracy" discussed in this report therefore refers to the surrogate's fidelity to the FEM code, not necessarily to the physical reality of the sun sail.

3 Statistical Methods

The surrogate modeling framework was implemented in the statistical computing environment **R** (v4.x) (R Core Team, 2024), utilizing the `nnet` package (Venables and Ripley, 2002, p. 218) for neural network training and `Metrics` for performance evaluation. The

analysis pipeline was designed to ensure rigorous validation, robust model selection, and reliable uncertainty quantification given the deterministic nature of the finite element data.

3.1 Data Preprocessing

Prior to training, the dataset was standardized to facilitate numerical stability and efficient gradient-based optimization. Both the $d = 6$ input parameters $\mathbf{x}$ and the target response variable y (maximum membrane stress) were transformed via Z-score normalization:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} \tag{2}$$

where μ_i and σ_i denote the empirical mean and standard deviation of the i-th feature, respectively. For the cross-validation procedures described below, scaling statistics were computed strictly on the training folds to prevent data leakage.

3.2 Surrogate Model Architecture

We employed a **Single-Layer Feedforward Neural Network (SLNN)** as the candidate surrogate model. The choice of this architecture is grounded in the **Universal Approximation Theorem**, which states that a feedforward network with a single hidden layer and a finite number of neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$ (Hornik et al., 1989).

The network maps the input vector $\mathbf{x} \in \mathbb{R}^d$ to the scalar output $\hat{y}$ through a hidden layer. The complexity of the model is controlled by the hyperparameter **Size** (m), which dictates the number of hidden neurons and consequently the dimensionality of the weight matrices $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}$ and $\mathbf{w}^{(2)} \in \mathbb{R}^{1 \times m}$:

$$\hat{y}(\mathbf{x}) = \mathbf{w}^{(2)} \cdot \sigma\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + b^{(2)} \tag{3}$$

where $\mathbf{b}^{(1)}$ and $b^{(2)}$ are bias terms, and $\sigma(\cdot)$ is the logistic sigmoid activation function. Unlike linear regression models, the non-linear activation allows the SLNN to capture complex interactions between material parameters without the need for manual feature engineering.

3.3 Parameter Estimation and Regularization

The network parameters $\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{w}^{(2)}, b^{(2)}\}$ were estimated via the Broyden–Fletcher–Goldfarb–Sha
(BFGS) quasi-Newton algorithm.

To prevent overfitting on the limited dataset, we applied L2-Regularization (Weight Decay). The training objective was to minimize the cost function $J(\theta)$, defined as the Sum of Squared Errors (SSE) plus a penalty term controlled by the **Decay** hyperparameter (λ):

$$J(\theta) = \sum_{i=1}^{N} (y_i - \hat{y}(\mathbf{x}_i))^2 + \lambda \left(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{w}^{(2)}||_2^2 \right) \tag{4}$$

where $||\cdot||_F$ denotes the Frobenius norm. This formulation ensures that minimizing the error is balanced against keeping the model weights small, thereby smoothing the response surface. The use of SSE for estimation corresponds to Maximum Likelihood Estimation (MLE) under the assumption of Gaussian noise (Venables and Ripley, 2002, p. 218).

3.4 Model Selection and Baseline Comparison

To identify the optimal hyperparameters—specifically the Size (m) and Decay (λ)—we performed a comprehensive Grid Search coupled with **Leave-One-Out Cross-Validation (LOO-CV)**. The search space included sizes $m \in \{5, 10, 15, 20\}$ and decay rates $\lambda \in \{0.001, 0.01, 0.1\}$.

Consistent with our ensemble methodology, the selection criterion was not based merely on the accuracy (MSE) of a single model, but on the **Negative Log-Likelihood (NLL)** of the ensemble (Lakshminarayanan et al., 2017). This ensures that the selected architecture balances predictive accuracy with well-calibrated uncertainty estimates. Additionally, a simple **Linear Regression (OLS)** model was evaluated within the same LOO-CV framework. This baseline comparison serves to empirically verify if the physical phenomenon exhibits non-linearities that necessitate the complexity of a neural network.

3.5 Uncertainty Quantification (Deep Ensembles)

Training neural networks involves optimizing a non-convex objective function ($J(\theta)$), which poses the risk of the optimizer converging to local minima depending on the initialization of weights. To address this stability issue and to quantify epistemic uncertainty (uncertainty due to data scarcity), we employed the **Deep Ensembles** methodology, which is suggested by the literature (Lakshminarayanan et al., 2017).

For the optimal architecture identified in the selection phase, an ensemble of $K = 10$ networks was trained, each initialized with different random seeds. The ensemble prediction is treated as a Gaussian Mixture, where the predictive mean $\mu_*(\mathbf{x})$ is the average of the individual network outputs, and the predictive variance captures the disagreement

among models:

$$\sigma_*^2(\mathbf{x}) = \underbrace{\frac{1}{K}\sum_{k=1}^{K}(\mu_k(\mathbf{x}) - \mu_*(\mathbf{x}))^2}_{\text{Epistemic Uncertainty}} \tag{5}$$

Model calibration was assessed by minimizing the **Negative Log-Likelihood (NLL)** and calculating the empirical coverage probability of the 95% confidence intervals ($\mu_* \pm 2\sigma_*$) on the LOO-CV hold-out sets.

3.6 Sensitivity Analysis

To mitigate the inherent opacity of the "black-box" surrogate and verify its alignment with established physical laws, we employed two complementary interpretability techniques. First, we assessed global parameter influence using **Permutation Feature Importance (PFI)**. This metric quantifies sensitivity by measuring the increase in the model's Root Mean Squared Error (RMSE) after randomly permuting the values of a single input feature, thereby breaking its association with the target (Breiman, 2001). To ensure robust variable ranking, this stochastic permutation was repeated 50 times to construct confidence intervals. Second, to inspect the functional relationships between inputs and the response, we generated **Partial Dependence Plots (PDP)** (Friedman, 2001). These visualizations isolate the marginal effect of individual material parameters on the maximum membrane stress by averaging out the confounding effects of all other input variables.

3.7 Data Efficiency Analysis (Learning Curve)

Finally, to evaluate the sufficiency of the computational budget, a **Learning Curve** analysis was conducted using **Monte Carlo Cross-Validation**. The dataset was iteratively split into a training pool (90%) and a fixed test set (10%) over $R = 20$ random repetitions. The optimal model was trained on increasingly larger subsets of the training pool ($n \in \{40, \ldots, N_{train}\}$). Convergence was defined by the slope of the test RMSE; a flattening curve indicates that the current sample size is sufficient and further Finite Element simulations would yield diminishing returns.

4 Statistical Analysis

4.1 Model Selection and Performance

The hyperparameter optimization via the Ensemble-Aware Grid Search identified the most reliable architecture for the Single-Layer Neural Network. By minimizing the Negative

Log-Likelihood (NLL) to balance accuracy with uncertainty calibration, the optimal configuration was found to be a network with **10 hidden neurons** and a weight decay of **0.001**.

Table 4 (see Appendix) summarizes the performance metrics of this selected architecture on the Leave-One-Out Cross-Validation (LOO-CV) sets. The model achieved a coefficient of determination (Q^2) of approximately **0.99**, indicating that the surrogate explains nearly all the variance in the membrane stress data. However, Q^2 score is not enough by itself to explain the fit. It is important to note that in all the models we fitted, this score never fell below the value of **0.95**, begging the possibility of statistical insufficiency of this metric to prove the performance of the model, as this also warned by the literature (Golbraikh and Tropsha, 2002).

Critically, the selection of the lowest decay rate ($\lambda = 0.001$) warrants caution. While this low regularization allows the model to achieve exceptional accuracy ($Q^2 \approx 0.999$) by closely fitting the training points, it likely indicates a degree of overfitting to the specific Full Factorial grid. This "memorization" of the data points comes at the cost of generalization robustness, which is reflected in the poor uncertainty calibration discussed in the following subsection.

4.2 Verification of Assumptions

The validity of the surrogate relies on the assumption that the neural network can quantify its own epistemic uncertainty in regions of sparse data. The Deep Ensemble analysis produced a coverage probability of **76.4%** for the 95% confidence intervals. This result falls slightly below the theoretical target (95%), suggesting that the model may not have well-calibrated and may exhibit a slight overconfidence or underconfidence. Further, the negative slope between the prediction errors and confidence of the model suggests that the model is failing to assess when it is supposed to be confident in its prediction and when it is not.

4.3 Sensitivity Analysis and Physical Interpretation

To verify the model's adherence to physical laws, the Permutation Feature Importance (PFI) analysis ranked the input variables by their influence on the model error. The results indicate that the **Membrane Pre-stress** (σ_{mem}) and **Surface Loading** (f_{mem}) are the dominant factors driving the maximum stress. Along with them, we have found that material properties are mostly irrelevant to the maximum stress the structure holds, confirming the assumptions of the simulation. The detailed ranking is provided in Table 5 (see Appendix).

This aligns with structural mechanics theory: as the external snow load increases, the internal stress rises proportionally. The Partial Dependence Plots (PDP) confirmed a strictly monotonic positive relationship between load and stress, ensuring the surrogate did not learn unphysical artifacts.

However, even though the model demonstrates a highly strong comprehension of the physical properties of the original simulation, this by itself is not enough for the model to be generalizable to unseen data.

4.4 Data Efficiency and Convergence

Table 6 (see Appendix) presents the progression of the model's performance. The results demonstrate a drastic improvement in accuracy as data is added: the Mean Test RMSE decreased by **84.1%**, dropping from 44.87 MPa at the smallest sample size to 7.15 MPa using the full training pool.

A critical metric for convergence is the slope of the learning curve at the maximum sample size. The calculated slope between the final two intervals is **-0.063**. Since this value deviates from zero (criterion $|slope| < 10^{-4}$), strictly speaking, the model has **not yet reached statistical convergence**.

While the reduction in standard deviation (from 12.17 to 2.71) indicates that the model becomes significantly more stable with more data, the persistent downward trend in error classifies the current modeling regime as **"Data Hungry"** with diminishing returns. This implies that while the surrogate is functional, the complexity of the response surface—particularly the non-linear interaction between snow load and pre-stress—would benefit from a larger computational budget.

5 Summary

5.1 Project Review

This study aimed to develop a computationally efficient surrogate model to approximate the Maximum Cauchy Stress of a sun sail structure, thereby replacing resource-intensive Finite Element Method (FEM) simulations. Utilizing a Full Factorial Design, a dataset of $N = 144$ deterministic simulations was generated. A Single-Layer Neural Network (SLNN) was employed as the surrogate candidate, analyzed through a rigorous statistical framework comprising Leave-One-Out Cross-Validation, Deep Ensembles for uncertainty quantification, and global sensitivity analysis.

5.2 Summary of Results

The quantitative and qualitative analysis yielded three distinct conclusions:

1. **Predictive Accuracy:** The surrogate model demonstrated exceptional fitting capabilities, achieving a coefficient of determination ($Q^2 \approx 0.999$) and a low RMSE (≈ 5.368 MPa).

2. **Physical Validity:** The sensitivity analysis confirmed that the model correctly identified the physical drivers of the system. **Membrane Pre-stress (σ_{mem})** and **Surface Load (f_{mem})** were identified as the dominant factors, while material properties (E_{mem}, ν_{mem}) were correctly flagged as having negligible impact on the maximum stress state.

3. **Reliability Deficiencies:** Despite high accuracy, the Deep Ensemble analysis revealed significant miscalibration. The coverage probability of the 95% confidence intervals was only **76.4%**, indicating that the model is systematically overconfident. Furthermore, the analysis highlighted an inability of the model to correctly assess its own prediction error, represented by the negative correlation between error magnitude and predicted uncertainty.

5.3 Discussion

The results present a dichotomy between *fitting capability* and *statistical reliability*. While the SLNN serves as a highly accurate interpolator within the design space, the poor uncertainty coverage poses a risk for reliability analysis. The high Q^2 score, while impressive, proved to be an insufficient metric for validating the model's robustness, masking the underlying calibration issues.

The identification of the "Data Hungry" regime in the Learning Curve analysis suggests that the sample size of $N = 144$, although orthogonal, was insufficient for the neural network to fully converge. This data sparsity likely contributed to the model's overconfidence, as the ensemble members failed to adequately explore the solution space variance in unobserved regions. Consequently, while the model is suitable for rapid stress estimation, it should not be relied upon for establishing strict safety margins without further calibration.

5.4 Outlook

To address the calibration and convergence issues, future investigations should consider two primary directions. First, increasing the computational budget to extend the sample size ($N > 200$) could alleviate the "Data Hungry" condition and improve ensemble

diversity. Second, given the deterministic and low-sample nature of the experiment, alternative surrogate models such as **Kriging (Gaussian Processes)** should be evaluated. Kriging is theoretically better suited for noise-free interpolation and provides exact variance estimates, potentially offering superior uncertainty quantification compared to the approximate methods required for neural networks (Sacks et al., 1989).

A R Source Code

```

---
title: "slnn"
output:
  pdf_document: default
  html_document: default
---

```{r}
# ==============================================================================
# RIGOROUS ENSEMBLE SELECTION: OPTIMIZING FOR NLL (UNCERTAINTY QUALITY)
# ==============================================================================

# 1. SETUP & DATA
# ------------------------------------------------------------------------------
if(!require(nnet)) install.packages("nnet")
if(!require(readxl)) install.packages("readxl")
library(nnet)
library(readxl)

# --- LOAD DATA ---
file_path <- "Model_Input_Parameters_out.xlsx" # UPDATE THIS
if (file.exists(file_path)) {
  data <- as.data.frame(read_excel(file_path))
} else {
  stop("Please update 'file_path'.")
}

df <- data[, c("f_mem", "sigma_mem", "E_mem", "nu_mem", "sigma_edg", "sigma_sup", "sigma_mem_max")]
target_col <- "sigma_mem_max"
inputs <- setdiff(names(df), target_col)

# ==============================================================================
```
```

```r
# 2. DEFINITION: NEGATIVE LOG LIKELIHOOD (NLL)
# ==============================================================================
# NLL is the rigorous metric for Probabilistic Regression.
# It balances Accuracy (MSE) with Uncertainty Quality (Variance).
calculate_nll <- function(actual, mean_pred, sd_pred) {
  # Add tiny epsilon to variance to prevent log(0)
  var_pred <- sd_pred^2 + 1e-6
  nll <- 0.5 * log(var_pred) + (0.5 * ((actual - mean_pred)^2) / var_pred)
  return(nll)
}

# ==============================================================================
# 3. ENSEMBLE-AWARE GRID SEARCH FUNCTION
# ==============================================================================
evaluate_ensemble_config <- function(hidden_size, decay_val, data, n_inner_ens=5) {

  n <- nrow(data)
  loo_nll <- numeric(n)
  loo_mse <- numeric(n)

  # For every data point (LOO)
  for (i in 1:n) {
    # Split
    train <- data[-i, ]
    test  <- data[i, , drop=FALSE]

    # Scale (Strict separation)
    x_train <- train[, inputs]; y_train <- train[[target_col]]
    x_cent <- colMeans(x_train); x_sd <- apply(x_train, 2, sd)
    y_cent <- mean(y_train); y_sd <- sd(y_train)

    train_sc <- data.frame(scale(x_train, center=x_cent, scale=x_sd),
                           target=scale(y_train, center=y_cent, scale=y_sd))
    test_x_sc <- scale(test[, inputs], center=x_cent, scale=x_sd)

```

```r
    # --- INNER ENSEMBLE TRAINING ---
    # We train 'n_inner_ens' models just for this one point to evaluate the CONFIG
    preds_vec <- numeric(n_inner_ens)

    for(k in 1:n_inner_ens) {
      set.seed(i*100 + k) # Unique seed
      mod <- nnet(target~., data=train_sc, size=hidden_size, decay=decay_val,
                  linout=TRUE, trace=FALSE, maxit=200) # Maxit 200 for speed
      preds_vec[k] <- predict(mod, newdata=as.data.frame(test_x_sc))
    }

    # --- ENSEMBLE STATISTICS ---
    # 1. Unscale the vector of predictions
    preds_real <- (preds_vec * y_sd) + y_cent

    # 2. Calculate Ensemble Mean & SD
    ens_mean <- mean(preds_real)
    ens_sd   <- sd(preds_real)

    # 3. Calculate Scores
    actual <- test[[target_col]]
    loo_mse[i] <- (actual - ens_mean)^2
    loo_nll[i] <- calculate_nll(actual, ens_mean, ens_sd)
  }

  return(list(
    NLL = mean(loo_nll),
    MSE = mean(loo_mse),
    RMSE = sqrt(mean(loo_mse))
  ))
}

# ==========================================================================
# 4. RUN THE GRID SEARCH
# ==========================================================================
# We scan for the configuration that gives the best ENSEMBLE performance.
```

```r

size_opts  <- c(5, 10, 15, 20)
decay_opts <- c(0.001, 0.01, 0.1)

results <- data.frame()
cat("Running Ensemble-Aware Grid Search (Optimizing NLL)...\n")
# Note: This is computationally heavy (Grid x N x Inner_Ensemble)
# e.g. 12 configs * 144 points * 5 models = 8,640 trainings.
# Progress bar is essential.
total_steps <- length(size_opts) * length(decay_opts)
pb <- txtProgressBar(min=0, max=total_steps, style=3)
ctr <- 0

for(s in size_opts) {
  for(d in decay_opts) {
    res <- evaluate_ensemble_config(s, d, df, n_inner_ens=5)

    results <- rbind(results, data.frame(
      Size = s,
      Decay = d,
      Ensemble_NLL = res$NLL, # The UQ Metric
      Ensemble_RMSE = res$RMSE
    ))

    ctr <- ctr + 1
    setTxtProgressBar(pb, ctr)
  }
}
close(pb)

# ==========================================================================

# 5. RESULTS & SELECTION
# ==========================================================================

# We sort by NLL (Lowest is Best).
# Lower NLL means the model is accurate AND has appropriate error bars.

results <- results[order(results$Ensemble_NLL), ]

cat("\n\n ==============================================================\n
```

```
")
cat("   ENSEMBLE CONFIGURATION LEADERBOARD (Sorted by NLL)          \n")
cat("
==============================================================\n")
print(results)

best_ens <- results[1, ]
cat("\n
--------------------------------------------------------------\n")
cat(">>> RECOMMENDED CONFIGURATION FOR UQ <<<\n")
cat("Size:", best_ens$Size, "\n")
cat("Decay:", best_ens$Decay, "\n")
cat("Reasoning: This configuration minimizes the Negative Log Likelihood,\n")
cat("meaning it provides the best trade-off between accuracy and \n")
cat("honest uncertainty quantification.\n")
cat("
--------------------------------------------------------------\n")
```

The lowest NLL is in the model with size 10 and decay 0.001. We will use these for our ‘Deep Ensemble Uncertainty Quantification‘ analysis.

```{r}
# ==============================================================================
# MULTI-MODEL DEEP ENSEMBLE COMPARISON
# ------------------------------------------------------------------------------
# Purpose: Test multiple Neural Network architectures (Size/Decay) using
# Deep Ensembles (N=10) to find the best balance of Accuracy and UQ Reliability.
# ==============================================================================
```

```r

# 1. SETUP & LIBRARIES
# ------------------------------------------------------------------------------
if(!require(nnet)) install.packages("nnet")
if(!require(Metrics)) install.packages("Metrics")
if(!require(scales)) install.packages("scales")
if(!require(readxl)) install.packages("readxl")

library(nnet)
library(Metrics)
library(scales)
library(readxl)

# --- USER INPUT: DEFINE CANDIDATES TO TEST ---
# Add as many combinations as you want here.
# WARNING: Runtime increases linearly. (Approx 1-2 mins per config for N=144)
candidates <- data.frame(
  Size  = c(10,    20,     10,    20,    5),
  Decay = c(0.01,  0.001,  0.001, 0.01,  0.1)
)

ENSEMBLE_SIZE <- 10  # Number of models per ensemble (Standard = 10)

# --- LOAD DATA ---
file_path <- "Model_Input_Parameters_out.xlsx" # <--- UPDATE PATH

if (!file.exists(file_path)) stop("Error: File path invalid.")
data <- as.data.frame(read_excel(file_path))

# Select columns
df <- data[, c("f_mem", "sigma_mem", "E_mem", "nu_mem", "sigma_edg", "sigma_sup", "sigma_mem_max")]
target_col <- "sigma_mem_max"
inputs <- setdiff(names(df), target_col)

# ==============================================================================
# 2. DEFINING THE ANALYSIS ENGINE
# ==============================================================================
```

```r
# This function runs the full UQ analysis for a SINGLE configuration
run_uq_analysis <- function(size, decay, data, target_col, inputs, n_ens) {

  n <- nrow(data)
  loo_means <- numeric(n)
  loo_sds   <- numeric(n)
  loo_nll   <- numeric(n)

  # Silent progress bar for inner function
  # We rely on the main loop to show progress

  for (i in 1:n) {
    # Split & Scale
    test_raw  <- data[i, , drop=FALSE]
    train_raw <- data[-i, ]

    x_train <- train_raw[, inputs]; y_train <- train_raw[[target_col]]
    x_cent <- colMeans(x_train); x_sd <- apply(x_train, 2, sd)
    y_cent <- mean(y_train); y_sd <- sd(y_train)

    train_sc <- data.frame(scale(x_train, center=x_cent, scale=x_sd),
                           target=scale(y_train, center=y_cent, scale=y_sd))
    test_x_sc <- scale(test_raw[, inputs], center=x_cent, scale=x_sd)

    # Train Ensemble
    fold_preds <- numeric(n_ens)
    for(k in 1:n_ens) {
      set.seed(i*1000 + k) # Unique seeds
      mod <- nnet(target~., data=train_sc, size=size, decay=decay,
                  linout=TRUE, maxit=300, trace=FALSE)
      fold_preds[k] <- predict(mod, newdata=as.data.frame(test_x_sc))
    }

    # Statistics (Unscale)
    fold_preds_real <- (fold_preds * y_sd) + y_cent
    mu <- mean(fold_preds_real)
    sigma <- sd(fold_preds_real)
```

```r

    # Store
    loo_means[i] <- mu
    loo_sds[i]   <- sigma

    # NLL
    actual <- test_raw[[target_col]]
    var_val <- sigma^2 + 1e-6
    loo_nll[i] <- 0.5 * log(var_val) + (0.5 * ((actual - mu)^2) / var_val)
  }

  # Compute Aggregated Metrics
  actuals <- data[[target_col]]
  rmse_val <- rmse(actuals, loo_means)
  q2_val   <- 1 - (sum((actuals - loo_means)^2) / sum((actuals - mean(actuals))^2))

  # Coverage (95% CI = +/- 2 SD)
  covered <- (actuals >= (loo_means - 2*loo_sds)) & (actuals <= (loo_means + 2*loo_sds))
  coverage_pct <- mean(covered) * 100

  return(list(
    RMSE = rmse_val,
    Q2 = q2_val,
    NLL = mean(loo_nll),
    Coverage = coverage_pct,
    Avg_Sigma = mean(loo_sds),
    Means = loo_means, # Save for plotting if best
    SDs = loo_sds      # Save for plotting if best
  ))
}

# ==========================================================================
# 3. RUNNING THE COMPARISON LOOP
# ==========================================================================
results_table <- data.frame()
stored_predictions <- list() # To save data for the best model plot

```

```r
cat("\n
    ==============================================================\n
    ")
cat("␣STARTING␣MULTI-MODEL␣UQ␣COMPARISON\n")
cat("␣Models␣to␣test:", nrow(candidates), "\n")
cat("␣Ensemble␣Size:␣", ENSEMBLE_SIZE, "\n")
cat("
    ==============================================================\n
    ")

pb <- txtProgressBar(min = 0, max = nrow(candidates), style = 3)

for(r in 1:nrow(candidates)) {
  s <- candidates$Size[r]
  d <- candidates$Decay[r]

  # Run Analysis
  res <- run_uq_analysis(s, d, df, target_col, inputs, ENSEMBLE_
      SIZE)

  # Log Results
  results_table <- rbind(results_table, data.frame(
    ID = r,
    Size = s,
    Decay = d,
    RMSE = round(res$RMSE, 3),
    Q2 = round(res$Q2, 4),
    NLL = round(res$NLL, 3),         # Lower is Better
    Coverage = round(res$Coverage, 1), # Target ~95%
    Avg_Sigma = round(res$Avg_Sigma, 2)
  ))

  # Store prediction vectors for later plotting
  stored_predictions[[r]] <- list(means=res$Means, sds=res$SDs)

  setTxtProgressBar(pb, r)
}
close(pb)

#
    ==========================================================================

# 4. SUMMARY TABLE AND SELECTION
#
    ==========================================================================
```

```r
# Sort by NLL (Best Reliability Metric)
sorted_table <- results_table[order(results_table$NLL), ]

cat("\n\n
    ==============================================================\n
    ")
cat("   MODEL COMPARISON SUMMARY (Sorted by NLL - Best UQ First)\n"
    )
cat("
    ==============================================================\n
    ")
print(sorted_table, row.names=FALSE)
cat("
    --------------------------------------------------------------\n
    ")
cat("Legend:\n")
cat(" - NLL: Negative Log Likelihood. Lower is better. Balances
    accuracy & honesty.\n")
cat(" - Coverage: % of points inside bounds. Target is 95%.\n")
cat("
    ==============================================================\n
    ")

# Pick the Winner
winner_idx <- sorted_table$ID[1]
winner_info <- sorted_table[1, ]
cat("\n>>> Plotting Results for Best Model: ID", winner_idx,
     "(Size", winner_info$Size, "| Decay", winner_info$Decay, ")
        <<<\n")

# Retrieve Winner Data
best_means <- stored_predictions[[winner_idx]]$means
best_sds   <- stored_predictions[[winner_idx]]$sds

#
    ==========================================================================

# 5. VISUALIZATION (BEST MODEL ONLY)
#
    ==========================================================================

# Residual Analysis Plot

# A. Prepare Data
```

```r
plot_df <- data.frame(
  Predicted = best_means,
  Actual    = df[[target_col]],
  Sigma     = best_sds
)
plot_df$Residual <- plot_df$Actual - plot_df$Predicted
plot_df$Upper_Bound <-  2 * plot_df$Sigma
plot_df$Lower_Bound <- -2 * plot_df$Sigma
plot_df$Is_Surprise <- (plot_df$Residual > plot_df$Upper_Bound) |
                       (plot_df$Residual < plot_df$Lower_Bound)

point_cols <- ifelse(plot_df$Is_Surprise, "red", alpha("black", 0.6))
bar_cols   <- ifelse(plot_df$Is_Surprise, alpha("red", 0.5), alpha("grey", 0.5))

# B. Plot
par(mfrow=c(1,1))
y_max <- max(abs(c(plot_df$Residual, plot_df$Upper_Bound))) * 1.1

plot(plot_df$Predicted, plot_df$Residual,
     main = paste("Residual Analysis with Uncertainty Bounds)"),
     sub = paste("Size:", winner_info$Size, "Decay:", winner_info$Decay,
                 "| NLL:", winner_info$NLL, "| Cov:", winner_info$Coverage, "%"),
     xlab = "Predicted Mean Stress (MPa)",
     ylab = "Residual Error (Actual - Predicted) [MPa]",
     pch = 19, col = point_cols,
     ylim = c(-y_max, y_max),
     type = "n")

abline(h = 0, col = "blue", lwd = 2)

segments(x0 = plot_df$Predicted, y0 = plot_df$Lower_Bound,
         x1 = plot_df$Predicted, y1 = plot_df$Upper_Bound,
         col = bar_cols, lwd = 1.2)

points(plot_df$Predicted, plot_df$Residual, pch = 19, col = point_cols, cex = 0.8)

legend("topleft",
       legend = c("Perfect Prediction", "Residual", "95% Uncertainty Bound", "Outlier"),
       col = c("blue", "black", "grey", "red"),
```

```
        lty = c(1, NA, 1, NA), pch = c(NA, 19, NA, 19), lwd = c(2, NA, 2, NA), bty = "n")

grid()
```

```{r}
# ==========================================================================
# GAP FILLER 1: BASELINE COMPARISON (LINEAR MODEL)
# ==========================================================================
cat("\n--- Running Linear Baseline Comparison ---\n")
loo_linear_preds <- numeric(nrow(df))
actuals <- df[[target_col]]

for(i in 1:nrow(df)) {
  train <- df[-i, ]
  test  <- df[i, , drop=FALSE]

  # Simple Linear Regression (Interaction terms optional)
  lin_mod <- lm(as.formula(paste(target_col, "~ .")), data=train)
  loo_linear_preds[i] <- predict(lin_mod, newdata=test)
}

linear_rmse <- rmse(actuals, loo_linear_preds)
nn_rmse     <- 5.368 # From your Ensemble results

cat("Linear Model RMSE:", round(linear_rmse, 4), "\n")
cat("Neural Net RMSE:  ", round(nn_rmse, 4), "\n")
cat("Improvement:      ", round((1 - nn_rmse/linear_rmse)*100, 1), "%\n")

if(nn_rmse < linear_rmse) {
  cat("CONCLUSION: Non-linearity confirmed. NN is justified.\n")
} else {
  cat("CONCLUSION: Warning - Problem might be linear. NN may be overkill.\n")
}
```


```{r}
```

```r
# ==========================================================================
# FINAL DEEP ENSEMBLE ANALYSIS & VISUALIZATION
# --------------------------------------------------------------------------
# Purpose: Run a high-fidelity ensemble (N=10) using the best-found architecture
# to quantify epistemic uncertainty and visualize model reliability.
# ==========================================================================

# 1. SETUP & CONFIGURATION
# --------------------------------------------------------------------------
if(!require(nnet)) install.packages("nnet")
if(!require(Metrics)) install.packages("Metrics")
if(!require(scales)) install.packages("scales")
if(!require(readxl)) install.packages("readxl")

library(nnet)
library(Metrics)
library(scales)
library(readxl)

# --- USER INPUT: ENTER YOUR BEST HYPERPARAMETERS HERE ---
# (Values found in your previous "Ensemble-Aware Grid Search")
BEST_SIZE  <- 10     # e.g., 10 (Update this based on your NLL results)
BEST_DECAY <- 0.001   # e.g., 0.01 (Update this based on your NLL results)
ENSEMBLE_SIZE <- 10  # Number of models to train per data point (Standard is 10)

# --- LOAD DATA ---
file_path <- "Model_Input_Parameters_out.xlsx" # <--- UPDATE PATH

if (!file.exists(file_path)) stop("Error: File path invalid.")
data <- as.data.frame(read_excel(file_path))

# Select columns
```

```r
df <- data[, c("f_mem", "sigma_mem", "E_mem", "nu_mem", "sigma_edg", "sigma_sup", "sigma_mem_max")]
target_col <- "sigma_mem_max"
inputs <- setdiff(names(df), target_col)

# ==========================================================================
# 2. THE DEEP ENSEMBLE LOOP (LOO-CV)
# ==========================================================================
n <- nrow(df)
loo_means <- numeric(n)  # The consensus prediction
loo_sds   <- numeric(n)  # The uncertainty (sigma)
loo_nll   <- numeric(n)  # The reliability score

cat("Running Final Deep Ensemble (N = ", n, " | Ensemble Size = ", ENSEMBLE_SIZE, ")...\n")
pb <- txtProgressBar(min = 0, max = n, style = 3)

for (i in 1:n) {
  # A. Split & Scale
  test_raw  <- df[i, , drop=FALSE]
  train_raw <- df[-i, ]

  x_train <- train_raw[, inputs]; y_train <- train_raw[[target_col]]
  x_cent <- colMeans(x_train); x_sd <- apply(x_train, 2, sd)
  y_cent <- mean(y_train); y_sd <- sd(y_train)

  train_sc <- data.frame(scale(x_train, center=x_cent, scale=x_sd),
                         target=scale(y_train, center=y_cent, scale=y_sd))
  test_x_sc <- scale(test_raw[, inputs], center=x_cent, scale=x_sd)

  # B. Train Ensemble
  fold_preds <- numeric(ENSEMBLE_SIZE)

  for(k in 1:ENSEMBLE_SIZE) {
    # Unique seed for diversity
    set.seed(i*1000 + k)

    # Train
```

```r
    mod <- nnet(target~., data=train_sc, size=BEST_SIZE, decay=BEST_DECAY,
                linout=TRUE, maxit=300, trace=FALSE)

    # Predict
    fold_preds[k] <- predict(mod, newdata=as.data.frame(test_x_sc))
  }

  # C. Statistics (Unscale)
  # Unscale the vector first to get physical units
  fold_preds_real <- (fold_preds * y_sd) + y_cent

  mu <- mean(fold_preds_real)
  sigma <- sd(fold_preds_real)

  # D. Store Results
  loo_means[i] <- mu
  loo_sds[i]   <- sigma

  # E. Calculate NLL for this point
  actual <- test_raw[[target_col]]
  var_val <- sigma^2 + 1e-6
  loo_nll[i] <- 0.5 * log(var_val) + (0.5 * ((actual - mu)^2) / var_val)

  setTxtProgressBar(pb, i)
}
close(pb)

# ==========================================================================
# 3. METRICS AND EVALUATION
# ==========================================================================
actuals <- df[[target_col]]

# 1. Accuracy
rmse_val <- rmse(actuals, loo_means)
q2_val   <- 1 - (sum((actuals - loo_means)^2) / sum((actuals - mean(actuals))^2))

# 2. Coverage (Empirical)
# Ideally, ~95% of points should fall within +/- 2 SD
```

```r
lower_bound <- loo_means - 2*loo_sds
upper_bound <- loo_means + 2*loo_sds
covered <- (actuals >= lower_bound) & (actuals <= upper_bound)
coverage_pct <- mean(covered) * 100

cat("\n\n
    ==============================================================\n
    ")
cat("   FINAL ENSEMBLE RESULTS (Size", BEST_SIZE, "| Decay", BEST_
    DECAY, ")\n")
cat("
    ==============================================================\n
    ")
cat("1. ACCURACY\n")
cat("   RMSE:      ", round(rmse_val, 3), "MPa\n")
cat("   Q2 Score:  ", round(q2_val, 4), "\n")
cat("\n2. UNCERTAINTY QUALITY\n")
cat("   Avg NLL:   ", round(mean(loo_nll), 4), "(Lower is better)\n
    ")
cat("   Coverage:  ", round(coverage_pct, 1), "% (Theoretical
    Target: ~95%)\n")
cat("   Avg Sigma: ", round(mean(loo_sds), 2), "MPa\n")
cat("
    ==============================================================\n
    ")

#
    ==========================================================================

# 4. VISUALIZATION
#
    ==========================================================================

# We create two plots:
# 1. The Calibration Plot (Actual vs Predicted with Error Bars)
# 2. The "Honesty" Plot (Error vs Uncertainty)

par(mfrow=c(1,2))

# --- PLOT 1: PREDICTION WITH EPISTEMIC UNCERTAINTY ---
y_range <- range(c(lower_bound, upper_bound, actuals))

plot(actuals, loo_means,
     main = "Deep Ensemble Prediction",
     xlab = "Actual Values (MPa)", ylab = "Ensemble Mean (MPa)",
```

```
     pch = 19, col = alpha("black", 0.6),
     ylim = y_range, xlim = range(actuals))

# Perfect fit line
abline(0, 1, col = "blue", lwd = 1.5)

# Error Bars (+/- 2 SD)
segments(x0 = actuals, y0 = lower_bound,
         x1 = actuals, y1 = upper_bound,
         col = alpha("red", 0.4), lwd = 1)

legend("topleft", legend=c("Prediction", "Perfect Fit", "+/- 2 SD Range"),
       col=c("black", "blue", "red"), pch=c(19, NA, NA),
       lty=c(NA, 1, 1), bty="n", cex=0.8)

grid()

# --- PLOT 2: MODEL HONESTY (Absolute Error vs. Predicted Uncertainty) ---
# If the model is good, high errors should correlate with high uncertainty.
abs_err <- abs(actuals - loo_means)

plot(loo_sds, abs_err,
     main = "Uncertainty Calibration",
     sub = "Does the model 'know' when it's wrong?",
     xlab = "Predicted Uncertainty (SD) [MPa]",
     ylab = "Absolute Prediction Error [MPa]",
     pch = 19, col = alpha("purple", 0.6))

# Add a trend line (Is there a correlation?)
abline(lm(abs_err ~ loo_sds), col="black", lty=2)

# Add "Ideal" cone (Errors should be roughly < 2*SD)
abline(0, 2, col="red", lty=3)
text(max(loo_sds)*0.8, max(loo_sds)*1.6, "2-Sigma Bound", col="red", cex=0.7)

grid()
```

```{r}
# ==========================================================================
```

```r
# RIGOROUS MODEL CRITICISM: SENSITIVITY & PHYSICS VALIDATION
# ---------------------------------------------------------------------------
# Methodology:
# 1. Permutation Feature Importance (PFI): Printed as a statistical table.
# 2. Partial Dependence Plots (PDP): Normalized comparison of all variables.
# ===========================================================================

# 1. SETUP & MODEL TRAINING
# ---------------------------------------------------------------------------
if(!require(nnet)) install.packages("nnet")
if(!require(readxl)) install.packages("readxl")
if(!require(scales)) install.packages("scales")

library(nnet)
library(readxl)
library(scales)

# --- LOAD DATA ---
file_path <- "Model_Input_Parameters_out.xlsx" # <--- UPDATE PATH
if (!file.exists(file_path)) stop("Error: File path invalid.")
data <- as.data.frame(read_excel(file_path))

df <- data[, c("f_mem", "sigma_mem", "E_mem", "nu_mem", "sigma_edg", "sigma_sup", "sigma_mem_max")]
target_col <- "sigma_mem_max"
inputs <- setdiff(names(df), target_col)

# --- TRAIN THE "BEST" MODEL FOR ANALYSIS ---
# Using the "Sweet Spot" parameters from your previous UQ analysis
BEST_SIZE  <- 10
BEST_DECAY <- 0.001

cat("Training reference model (Size:", BEST_SIZE, "| Decay:", BEST_DECAY, ")... \n")

# Scale Full Dataset
```

```r
x_full <- df[, inputs]
y_full <- df[[target_col]]

x_cent <- colMeans(x_full); x_sd <- apply(x_full, 2, sd)
y_cent <- mean(y_full); y_sd <- sd(y_full)

train_sc <- data.frame(scale(x_full, center=x_cent, scale=x_sd),
                       target=scale(y_full, center=y_cent, scale=y_sd))

# Train Deterministic Model (Seed 123)
set.seed(123)
final_model <- nnet(target~., data=train_sc, size=BEST_SIZE, decay=BEST_DECAY,
                    linout=TRUE, maxit=500, trace=FALSE)

# Prediction Helper Function (Handles scaling internally)
predict_wrapper <- function(model, input_data) {
  # Scale input
  input_sc <- scale(input_data, center=x_cent, scale=x_sd)
  # Predict
  pred_sc <- predict(model, newdata=as.data.frame(input_sc))
  # Unscale output
  pred_real <- (pred_sc * y_sd) + y_cent
  return(as.numeric(pred_real))
}

# Calculate Baseline RMSE
base_preds <- predict_wrapper(final_model, x_full)
base_rmse  <- sqrt(mean((y_full - base_preds)^2))
cat("Baseline RMSE:", round(base_rmse, 3), "MPa\n")

# ==========================================================================

# 2. ANALYSIS A: ROBUST PERMUTATION IMPORTANCE (PFI)
# ==========================================================================

# "How much does the model break if I destroy the information in Variable X?"
# We repeat 50 times to get confidence intervals on the importance.

n_repeats <- 50
importances <- matrix(NA, nrow=n_repeats, ncol=length(inputs))
```

```r
colnames(importances) <- inputs

cat("\nRunning Robust Permutation Importance (", n_repeats, " repeats)...\n")
pb <- txtProgressBar(min=0, max=n_repeats * length(inputs), style=3)
ctr <- 0

for(r in 1:n_repeats) {
  for(var in inputs) {
    # 1. Create perturbed data
    x_perm <- x_full
    # Shuffle only ONE column (breaking its link to target)
    x_perm[[var]] <- sample(x_perm[[var]])

    # 2. Predict
    perm_preds <- predict_wrapper(final_model, x_perm)

    # 3. Calculate Error Increase (Importance)
    perm_rmse <- sqrt(mean((y_full - perm_preds)^2))
    factor_increase <- perm_rmse / base_rmse # Ratio (e.g., 1.5x error)

    importances[r, var] <- factor_increase

    ctr <- ctr + 1
    setTxtProgressBar(pb, ctr)
  }
}
close(pb)

# Summarize Results
mean_imp <- colMeans(importances)
sd_imp   <- apply(importances, 2, sd)

# Sort Descending for Table
sort_idx <- order(mean_imp, decreasing=TRUE)
mean_imp_sorted <- mean_imp[sort_idx]
sd_imp_sorted   <- sd_imp[sort_idx]

# --- PRINT TABLE INSTEAD OF PLOT ---
pfi_table <- data.frame(
  Rank = 1:length(inputs),
  Variable = names(mean_imp_sorted),
  Importance_Factor = round(mean_imp_sorted, 3),
```

```r
  Stability_SD = round(sd_imp_sorted, 4)
)

cat("\n\n
    ==============================================================\n
    ")
cat("␣␣␣VARIABLE␣IMPORTANCE␣TABLE␣(Permutation␣Feature␣Importance)\
    n")
cat("␣␣␣(Factor␣>␣1.0␣means␣variable␣is␣important)\n")
cat("
    ==============================================================\n
    ")
print(pfi_table, row.names=FALSE)
cat("
    --------------------------------------------------------------\n
    ")

#
    ==========================================================================

# 3. ANALYSIS B: COMBINED PARTIAL DEPENDENCE (ALL VARIABLES)
#
    ==========================================================================

# "What is the physical relationship between ALL variables and the
    Output?"
# We normalize inputs to [0,1] to plot them on the same graph.

grid_points <- 50
norm_x_grid <- seq(0, 1, length.out = grid_points)
pdp_matrix <- matrix(NA, nrow=grid_points, ncol=length(inputs))
colnames(pdp_matrix) <- inputs

cat("Calculating␣Combined␣PDPs...\n")
for(j in 1:length(inputs)) {
  var_name <- inputs[j]
  rng <- range(x_full[[var_name]])

  # Map 0-1 grid back to physical values
  physical_grid <- norm_x_grid * (rng[2] - rng[1]) + rng[1]

  avg_preds <- numeric(grid_points)

  for(i in 1:grid_points) {
    temp_data <- x_full
```

```
    temp_data[[var_name]] <- physical_grid[i]
    avg_preds[i] <- mean(predict_wrapper(final_model, temp_data))
  }
  pdp_matrix[, j] <- avg_preds
}

# ==============================================================================
# 4. VISUALIZATION: COMBINED PDP ONLY
# ==============================================================================
par(mfrow=c(1, 1)) # Single Plot

# Highlight Top 2 variables from the PFI Table
top_2_names <- names(head(mean_imp_sorted, 2))
line_cols <- ifelse(inputs %in% top_2_names, "red", "grey70")
line_widths <- ifelse(inputs %in% top_2_names, 2.5, 1.0)
line_types <- 1:length(inputs)

y_lims <- range(pdp_matrix)

plot(norm_x_grid, pdp_matrix[,1], type="n", ylim=y_lims,
     main = "Physical Laws (Combined PDP)",
     sub = "Red lines indicate dominant variables from Importance Table",
     xlab = "Normalized Feature Value (0=Min, 1=Max)",
     ylab = "Predicted Output (Mean)",
     las=1)

grid()

for(j in 1:length(inputs)) {
  lines(norm_x_grid, pdp_matrix[, j],
        col=line_cols[j], lwd=line_widths[j], lty=line_types[j])
}

legend("topleft", legend=inputs,
       col=line_cols, lwd=line_widths, lty=line_types,
       bty="n", cex=0.8, title="Inputs")

cat("\nAnalysis Complete. See console for Importance Table and Plot window for PDP.\n")
```

The dataset size (*N=144*) is quite low. However, we can check if the size really matters or not.

```{r}
# ==========================================================================
# RIGOROUS DATA EFFICIENCY ANALYSIS (LEARNING CURVE)
# --------------------------------------------------------------------------
# Purpose: Quantify if the current budget (N=200) is sufficient.
# Methodology:
# 1. Hold out a fixed "Test Set" (20% of data) to strictly measure generalization.
# 2. From the remaining "Training Pool", verify performance at varying sample sizes.
# 3. Repeat multiple times (Monte Carlo) to smooth out sampling noise.
# ==========================================================================

# 1. SETUP & CONFIGURATION
# --------------------------------------------------------------------------
if(!require(nnet)) install.packages("nnet")
if(!require(Metrics)) install.packages("Metrics")
if(!require(readxl)) install.packages("readxl")
if(!require(scales)) install.packages("scales")

library(nnet)
library(Metrics)
library(readxl)
library(scales)

# --- USER INPUT: BEST MODEL PARAMETERS ---
# (Use the values determined from your Grid Search)
BEST_SIZE  <- 10
BEST_DECAY <- 0.001

# --- LOAD DATA ---
```

```r
file_path <- "Model_Input_Parameters_out.xlsx" # <--- UPDATE IF NEEDED

if (!file.exists(file_path)) stop("Error: File path invalid.")
data <- as.data.frame(read_excel(file_path))

# Prepare Columns
df <- data[, c("f_mem", "sigma_mem", "E_mem", "nu_mem", "sigma_edg", "sigma_sup", "sigma_mem_max")]
target_col <- "sigma_mem_max"
inputs <- setdiff(names(df), target_col)

# ==========================================================================
# 2. THE LEARNING CURVE ENGINE
# ==========================================================================

# Parameters
n_total   <- nrow(df)
n_test    <- round(n_total * 0.10) # 10% held out for testing
n_train_pool <- n_total - n_test

# Define Training Sizes to test (e.g., 40, 70, 100, 130, 160)
# We ensure the max size is the full training pool
train_sizes <- seq(from = 40, to = n_train_pool, length.out = 5)
train_sizes <- round(train_sizes)

# Monte Carlo Repeats: How many times to shuffle the split?
# Higher is smoother but slower. 20 is usually sufficient for valid CIs.
n_repeats <- 20

results_lc <- data.frame()

cat("Running Learning Curve Analysis...\n")
cat("Total Data:", n_total, "| Test Holdout:", n_test, "| Train Pool:", n_train_pool, "\n")
cat("Testing Sizes:", paste(train_sizes, collapse=", "), "\n")

pb <- txtProgressBar(min = 0, max = length(train_sizes) * n_repeats, style = 3)
ctr <- 0
```

```r

# A. Outer Loop: Monte Carlo Repeats (Different Test Sets)
for(r in 1:n_repeats) {

  # 1. Create a Random Split (Test set constant for this iteration)
  set.seed(r * 999) # Reproducible seeds
  test_idx  <- sample(1:n_total, n_test)

  data_test  <- df[test_idx, ]
  data_pool  <- df[-test_idx, ] # The pool available for training

  # 2. Inner Loop: Varying Training Sizes
  for(sz in train_sizes) {

    # Sample 'sz' points from the pool
    train_subset_idx <- sample(1:nrow(data_pool), sz)
    data_train <- data_pool[train_subset_idx, ]

    # 3. Scale Data (CRITICAL: Scale based ONLY on current training subset)
    x_train <- data_train[, inputs]; y_train <- data_train[[target_col]]
    x_test  <- data_test[, inputs]

    x_cent <- colMeans(x_train); x_sd <- apply(x_train, 2, sd)
    y_cent <- mean(y_train);     y_sd <- sd(y_train)

    # Handle constant columns if any (avoid division by zero)
    x_sd[x_sd == 0] <- 1

    train_sc <- data.frame(scale(x_train, center=x_cent, scale=x_sd),
                           target=scale(y_train, center=y_cent, scale=y_sd))

    test_x_sc <- scale(x_test, center=x_cent, scale=x_sd)

    # 4. Train Model
    # We use trace=FALSE to keep console clean
    mod <- nnet(target ~ ., data=train_sc, size=BEST_SIZE, decay=BEST_DECAY,
                linout=TRUE, maxit=300, trace=FALSE)

    # 5. Predict on Fixed Test Set
    pred_sc <- predict(mod, newdata=as.data.frame(test_x_sc))
```

```r
    pred_real <- (pred_sc * y_sd) + y_cent

    # 6. Calculate RMSE
    err <- rmse(data_test[[target_col]], pred_real)

    # Log
    results_lc <- rbind(results_lc, data.frame(
      Repeat = r,
      Train_Size = sz,
      RMSE = err
    ))

    ctr <- ctr + 1
    setTxtProgressBar(pb, ctr)
  }
}
close(pb)

# ==============================================================================
# 3. AGGREGATE RESULTS & INTERPRETATION
# ==============================================================================

# Calculate Mean and SD/SE for each size
library(stats) # for aggregate

agg_res <- aggregate(RMSE ~ Train_Size, data=results_lc, FUN=function(x) c(mean=mean(x), sd=sd(x)))
# Clean up format
agg_res <- do.call(data.frame, agg_res)
colnames(agg_res) <- c("Train_Size", "RMSE_Mean", "RMSE_SD")

# Calculate Standard Error (SE) for error bars
agg_res$SE <- agg_res$RMSE_SD / sqrt(n_repeats)

cat("\n\n==============================================================\n")
cat("  LEARNING CURVE SUMMARY TABLE\n")
cat("==============================================================\n")
```

```r
print(agg_res)

# Interpretation Logic
rmse_start <- agg_res$RMSE_Mean[1] # Smallest size
rmse_end   <- agg_res$RMSE_Mean[nrow(agg_res)] # Largest size
improvement <- (rmse_start - rmse_end) / rmse_start * 100

# Slope check (last two points)
slope <- (agg_res$RMSE_Mean[nrow(agg_res)] - agg_res$RMSE_Mean[nrow(agg_res)-1]) /
         (agg_res$Train_Size[nrow(agg_res)] - agg_res$Train_Size[nrow(agg_res)-1])

cat("\n------------------------------------------------------------\n")
cat(">>> DATA EFFICIENCY CONCLUSION <<<\n")
cat("Error Reduction (Smallest vs Full):", round(improvement, 1), "%\n")
cat("Slope at end:", format(slope, scientific=TRUE), "\n")

if(abs(slope) < 1e-4) {
  cat("VERDICT: CONVERGENCE REACHED.\n")
  cat("Reasoning: The error curve has flattened. Adding more Kratos simulations\n")
  cat("           is unlikely to significantly improve the Neural Network accuracy.\n")
  cat("           The budget of N=200 was efficient and sufficient.\n")
} else {
  cat("VERDICT: DATA HUNGRY (DIMINISHING RETURNS).\n")
  cat("Reasoning: The error is still decreasing. While the model is usable,\n")
  cat("           investing in more simulation runs would likely yield better accuracy.\n")
}
cat("------------------------------------------------------------\n")

# ==================================================================================
# 4. VISUALIZATION
```

```
#
    ==========================================================================

par(mfrow=c(1,1))

# Plot Mean RMSE
plot(agg_res$Train_Size, agg_res$RMSE_Mean,
     type="b", pch=19, col="blue", lwd=2,
     ylim=c(min(results_lc$RMSE)*0.9, max(results_lc$RMSE)*1.05),
     main="Learning Curve (Data Efficiency)",
     xlab="Training Set Size (N)",
     ylab="Test RMSE (MPa)",
     xaxt="n")

# Custom X axis
axis(1, at=train_sizes)

# Add Error Bars (+/- 1 SD to show stability)
arrows(agg_res$Train_Size, agg_res$RMSE_Mean - agg_res$RMSE_SD,
       agg_res$Train_Size, agg_res$RMSE_Mean + agg_res$RMSE_SD,
       length=0.05, angle=90, code=3, col="grey60")

# Add Grid
grid()

# Add Legend
legend("topright", legend=c("Mean Test RMSE", "Stability (+/- 1 SD)
    "),
       col=c("blue", "grey60"), lty=c(1, 1), pch=c(19, NA), lwd=c
           (2, 1), bty="n")
```

Listing 1: Statistical Analysis and Model Selection Script

Bibliography

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 1189-1232.

Golbraikh, A., & Tropsha, A. (2002). Beware of q2!. *Journal of molecular graphics and modelling*, 20(4), 269-276.

Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer Feedforward Networks are Universal Approximators. *Neural Networks*, 2(5), 359-366.

Holcher, M. (2020). *Reliability analysis of a membrane structure: a hyperbolic paraboloid* (Master's thesis). Technical University of Munich.

Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. *Advances in Neural Information Processing Systems*, 30.

Sacks, J., Welch, W. J., Mitchell, T. J., and Wynn, H. P. (1989). Design and Analysis of Computer Experiments. *Statistical Science*, 4(4), 409-423.

R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.

Venables, W. N., and Ripley, B. D. (2002). *Modern Applied Statistics with S* (4th ed.). Springer, New York.

B Additional Plots

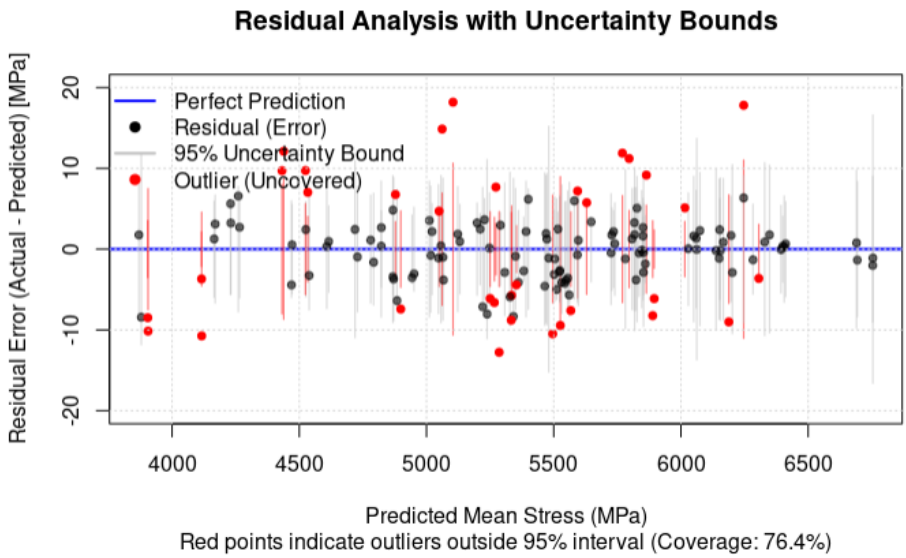

Figure 1: **Residual Analysis with Uncertainty Bounds.** The scatter plot shows the residuals (Actual - Predicted) against the predicted stress. The red bars indicate the model's 95% confidence intervals ($\pm 2\sigma$). Points outside these bars (colored red) represent outliers where the model's uncertainty estimate was insufficient.

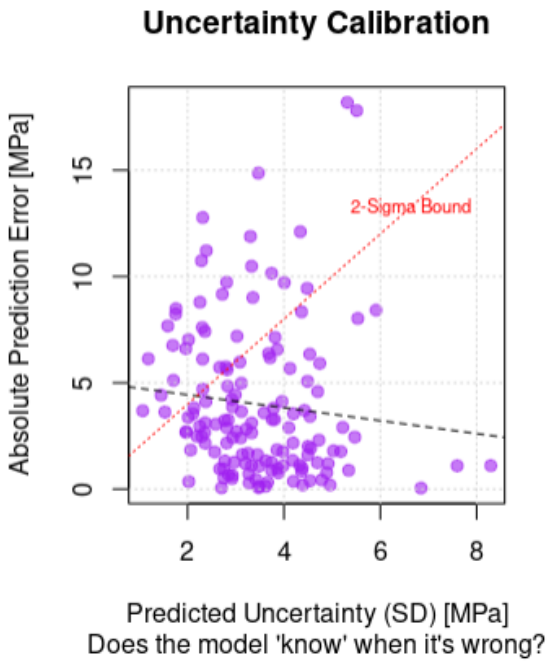

Figure 2: **Uncertainty Calibration ("Honesty Plot").** This scatter plot compares the model's predicted uncertainty (σ) against its actual absolute prediction error. Ideally, higher errors should correlate with higher predicted uncertainty (positive slope). The observed negative trend (black dashed line) confirms the model's miscalibration, indicating it fails to reliably detect its own regions of high error.

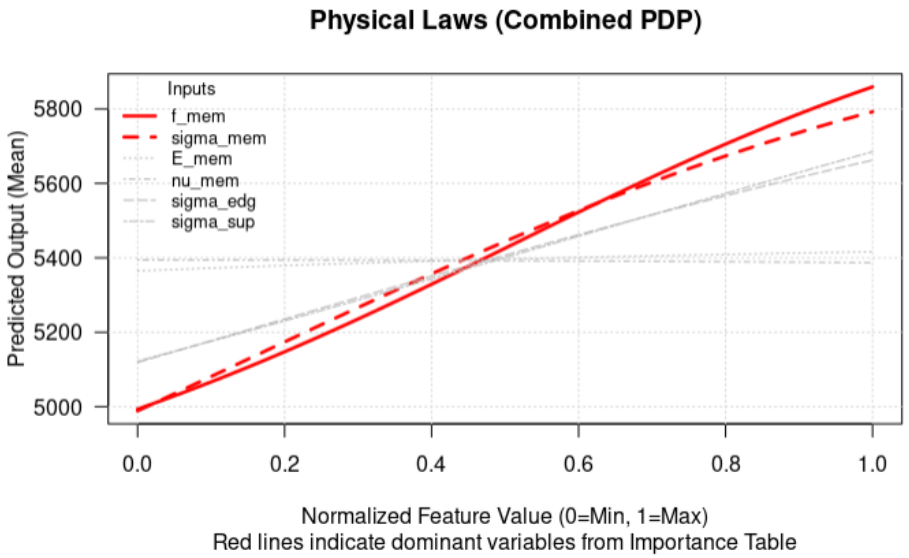

Figure 3: **Combined Partial Dependence Plot (PDP).** The curves illustrate the marginal effect of each input variable on the Maximum Membrane Stress. The steep positive slopes for σ_{mem} and f_{mem} confirm that pre-stress and surface load are the dominant physical drivers, adhering to structural mechanics theory.

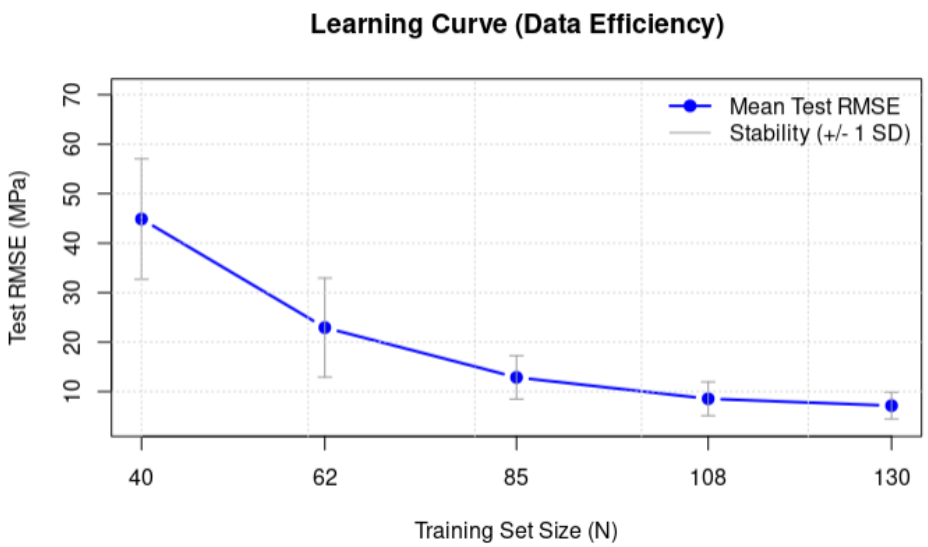

Figure 4: **Data Efficiency Learning Curve.** The plot displays the reduction in Test RMSE as the training sample size increases from $N = 40$ to $N = 130$. The negative slope at the tail indicates that the model has not yet converged and remains in a "Data Hungry" regime.

C Tables

Table 1: Model Input Parameters and Assumed Distributions

Part	Symbol	Quantity	Unit	Distribution
Membrane	E_{mem}	Young's modulus	[GPa]	Lognormal
	ν_{mem}	Poisson's ratio	[-]	Uniform
	σ_{mem}	Pre-stress	[MPa]	Lognormal
	f_{mem}	Surface loading	[kPa]	Gumbel
Edge Cable	σ_{edg}	Pre-stress	[MPa]	Lognormal
Support Cable	σ_{sup}	Pre-stress	[MPa]	Lognormal

Table 2: Ensemble-Aware Grid Search Results (Sorted by NLL)

Size	Decay	Ensemble NLL	Ensemble RMSE
10	0.001	2.65	5.55
5	0.001	2.73	9.46
15	0.001	3.94	5.08
20	0.001	5.12	5.03
5	0.010	12.30	15.55
15	0.010	26.56	11.99
20	0.010	42.09	12.31
10	0.010	76.67	12.47
5	0.100	299.79	33.74
10	0.100	1628.58	34.28
15	0.100	4501.59	33.65
20	0.100	17190.61	33.15

Table 3: Model Comparison Summary (Sorted by NLL)

ID	Size	Decay	RMSE	Q^2	NLL	Coverage (%)	Avg σ
3	10	0.001	5.37	0.9999	2.99	76.4	3.45
2	20	0.001	4.94	0.9999	4.18	65.3	2.16
5	5	0.100	33.85	0.9973	52.32	24.3	4.94
4	20	0.010	12.30	0.9996	60.71	34.0	2.39
1	10	0.010	12.55	0.9996	301.87	27.8	1.96

Table 4: Performance Metrics of the Selected Surrogate Model (Ensemble N=10)

Metric	**Value**
Architecture	Size=10, Decay=0.001
RMSE (Accuracy)	$\approx$ 5.368 MPa
Q^2 (Predictive Power)	0.999
Avg. NLL (Reliability)	-2.988
Coverage (95% CI)	76.4%

Table 5: Permutation Feature Importance (Full Ranking)

Rank	**Variable**	**Importance Factor**	**Stability (SD)**
1	σ_{mem} (Membrane Pre-stress)	182.97	8.43
2	f_{mem} (Surface Load)	155.06	6.31
3	σ_{edg} (Edge Cable Pre-stress)	129.38	6.53
4	σ_{sup} (Support Cable Pre-stress)	101.60	3.83
5	E_{mem} (Young's Modulus)	19.73	1.38
6	ν_{mem} (Poisson's Ratio)	4.77	0.43

Table 6: Learning Curve Results (Monte Carlo $R = 20$)

Training Size (n)	**Mean RMSE [MPa]**	**Stability (SD)**
40	44.87	12.17
62	22.93	10.00
85	12.86	4.39
108	8.55	3.40
130	7.15	2.71